

Lecture 18: Closures, Higher-Order Functions, and the this Keyword

Scope and Closures:

- **Variable Scope:** In JavaScript, `let` and `const` respect block scope, while `var` only respects functional scope.
- **Lexical Environment:** When a function requires a variable that is not defined within its immediate scope, it looks "outward" into its surrounding lexical environment to find it.
- **What is a Closure?** A closure is a function that remembers variables from its outer scope even after the outer function has finished executing.
- **Misconception:** Because closures retain variables in memory after execution, they are sometimes mistaken for memory leaks or bugs. In reality, they are an intentional and powerful core feature of JavaScript.
- **Use Case (Data Privacy):** Closures are heavily used to encapsulate data and create private variables. This allows variables to be modified securely through returned methods while preventing accidental, direct manipulation from the outside.

```
function bank() {  
  let balance = 2000;  
  
  return {  
    credit: function(amount) {  
      if (typeof amount === "number")  
        balance += amount;  
    },  
    debit: function(amount) {  
      if (typeof amount === "number" && amount <= balance)  
        balance -= amount;  
    },  
    checkBalance: function() {  
      console.log(balance);  
    }  
  };  
}
```

```
const user = bank();  
user.checkBalance()
```

Higher-Order Functions (HOF):

A Higher-Order Function is defined as a function that either accepts one or more functions as arguments, or returns a function as its result.

```
function increment(amount) {  
  return function mul(num) {  
    console.log(num * amount);  
  };  
}
```

```
increment(30)(10);
```

The this Keyword:

The behavior of this changes depending on the execution environment (Browser vs. Node.js) and the strictness of the code.

- **Global Scope:** In a browser, logging this returns the window object, while in Node.js, it returns an empty object {}. (To standardize global context across environments, JavaScript introduced globalThis).
- **Inside an Object Method:** Whether the code is in Strict or Non-Strict mode, this always points to the specific object that invoked the method.
- **Inside a Regular Function:**
 - **In a Non-Strict Mode:** this defaults to pointing at the global object.
 - **Strict Mode ("use strict"):** this returns undefined to prevent accidental global modifications.

```
const user = {  
  name: "Rohit",  
  greet: function() {  
    console.log(this);  
  }  
};  
  
user.greet();
```

Context Manipulation: call, apply, and bind:

Before the introduction of Classes in JavaScript, if developers had hundreds of object instances, they wouldn't duplicate the exact same method inside every single one. Instead, they created a single, standalone function and manually attached it to different objects using context manipulation.

- **call():** Invokes the function immediately, setting this to the provided object. Additional arguments are passed individually, separated by commas.
- **apply():** Functions exactly like call(), but additional arguments must be passed together as a single array.
- **bind():** Does not execute the function immediately. Instead, it binds this to the object and returns a new function reference that can be executed later.

```
const user = {  
  name: "Rohit",  
  age: 20,  
  amount: 500  
};  
  
function increment(umar, paisa) {  
  this.age = umar;  
  this.amount += paisa;  
  console.log(this);  
}  
  
increment.call(user, 30, 300);  
  
increment.apply(user, [30, 300]);  
  
const ref = increment.bind(user, 30, 300);  
ref();
```

Arrow Functions and Lexical this:

- **No Internal this:** Unlike regular functions, arrow functions do not have their own this context.
- **Borrowing Context:** They borrow the this value from their surrounding lexical environment (the outer scope).
- **The Object Pitfall:** Because standard objects are just data structures and do not create a lexical scope, an arrow function defined directly inside an object will look past the object and borrow this from the global scope.
- **Practical Use:** Arrow functions are highly useful inside callbacks (like setInterval nested inside an object method) because they prevent this from resetting to the global object, correctly maintaining the outer method's context.

```
const watch = {  
  timer: 0,  
  stopWatch: function() {  
    setInterval(() => {  
      this.timer++;  
      console.log(this.timer);  
    }, 1000);  
  }  
};  
  
watch.stopWatch();
```